

Linked List Part 2: Medium Singly Linked List Problems

Node Structures

```
#include <bits/stdc++.h>
using namespace std;

struct Node {
    int data;
    Node* next;

    Node(int data1) {
        data = data1;
        next = nullptr;
    }

    Node(int data1, Node* next1) {
        data = data1;
        next = next1;
    }
};

struct DNode {
    int data;
    DNode* prev;
    DNode* next;

    DNode(int data1) {
        data = data1;
        prev = nullptr;
        next = nullptr;
    }
};
```

10. Middle of a LinkedList

Problem

Return the middle node of a linked list. For even length, return the second middle.

Brute Approach

Count nodes first, then move to the middle.

```
Node* middleNodeBrute(Node* head) {  
    // Hint: first pass length, second pass n/2 +1 moves.  
    int n = 0;  
    Node* temp = head;  
  
    while (temp != nullptr) {  
        n++;  
        temp = temp->next;  
    }  
  
    int middle = (n / 2)+1;  
    temp = head;  
    while (temp!=nullptr) {  
        middle--;  
        if(middle==0){  
            break;  
        }  
        temp = temp->next;  
    }  
  
    return temp;  
}
```

Time Complexity: $O(n + n/2)$

Space Complexity: $O(1)$

Better

No separate better approach needed.

Optimal Approach

Use slow and fast pointers. Fast moves two steps; slow moves one step.

```
Node* middleNodeOptimal(Node* head) {  
    // Hint: fast jab end par hoga, slow middle par hoga.  
    Node* slow = head;  
    Node* fast = head;  
  
    while (fast != nullptr && fast->next != nullptr) {  
        slow = slow->next;  
        fast = fast->next->next;  
    }  
  
    return slow;  
}
```

Time Complexity: $O(n)$

Space Complexity: $O(1)$

11. Reverse a LinkedList: Iterative

Problem

Reverse a singly linked list using iteration.

Brute Approach

Store values in stack and rewrite node data.

```

Node* reverseSLLBrute(Node* head) {
    // Hint: stack se values reverse order mein milengi, links change nahi honge.
    stack<int> st;
    Node* temp = head;

    while (temp != nullptr) {
        st.push(temp->data);
        temp = temp->next;
    }

    temp = head;
    while (temp != nullptr) {
        temp->data = st.top();
        st.pop();
        temp = temp->next;
    }

    return head;
}

```

Time Complexity: $O(2n)$

Space Complexity: $O(n)$

Better

No separate better approach needed.

Optimal Approach

Reverse links using `prev`, `temp`, and `front`.

```

Node* reverseSLLiterativeOptimal(Node* head) {
    // Hint: front ko save karo, temp->next ko prev par reverse karo.
    Node* prev = nullptr;
    Node* temp = head;

    while (temp != nullptr) {
        Node* front = temp->next;
        temp->next = prev;
        prev = temp;
        temp = front;
    }

    return prev;
}

```

Time Complexity: $O(n)$

Space Complexity: $O(1)$

12. Reverse a LinkedList: Recursive

Problem

Reverse a linked list using recursion.

Brute / Better

The stack-value method from Problem 11 is brute. Recursion below is the link-changing approach.

Optimal Approach

Reverse the rest of the list, then attach current node after its next node.

```

Node* reverseSLLRecursiveOptimal(Node* head) {
    // Hint: recursion last node ko new head bana deti hai.
    if (head == nullptr || head->next == nullptr) return head;

    Node* newHead = reverseSLLRecursiveOptimal(head->next);
    Node* front = head->next;
    front->next = head;
    head->next = nullptr;

    return newHead;
}

```

Time Complexity: $O(n)$

Space Complexity: $O(n)$, due to recursion stack

13. Detect a Loop in LL

Problem

Check whether the linked list contains a cycle.

Brute Approach

Store visited node addresses in a hash set.

```

bool detectLoopBrute(Node* head) {
    // Hint: same node address dobara mila to cycle hai.
    unordered_map<Node*, int> mp;
    Node* temp = head;

    while (temp != nullptr) {
        if (mp.find(temp)) return true; // it means we are visiting twice aur hum sirf
        loop me hi twice visit kr sakte hai
        mp[temp]++;
        temp = temp->next;
    }

    return false;
}

```

Time Complexity: $O(n)$ average

Space Complexity: $O(n)$

Better

No separate better approach needed.

Optimal Approach

Use Floyd's cycle detection: slow moves one step, fast moves two steps.

```
bool detectLoopOptimal(Node* head) {  
    // Hint: cycle hai to fast aur slow kabhi na kabhi milenge.  
    Node* slow = head;  
    Node* fast = head;  
  
    while (fast != nullptr && fast->next != nullptr) {  
        slow = slow->next;  
        fast = fast->next->next;  
  
        if (slow == fast) return true;  
    }  
  
    return false;  
}
```

Time Complexity: $O(n)$

Space Complexity: $O(1)$

14. Find the Starting Point in LL

Problem

If a cycle exists, return the node where the cycle begins.

Brute Approach

Store visited nodes. First repeated node is the starting point.

```

Node* startingPointBrute(Node* head) {
    // Hint: first repeated address is loop ka entry point.
    unordered_map<Node*, int> mp;
    Node* temp = head;

    while (temp != nullptr) {
        if (mp.find(temp)) return temp; // it means we are visiting it for the first time
        // me twice aur hum sirf loop me hi twice visit kr sakte hai
        mp[temp]++;
        temp = temp->next;
    }

    return nullptr;
}

```

Time Complexity: $O(n)$ average

Space Complexity: $O(n)$

Better

No separate better approach needed.

Optimal Approach

First detect meeting point using slow-fast. Then put slow at head and move both one step.


```

Node* startingPointOptimal(Node* head) {
    // Hint: meeting ke baad slow=head; slow-fast one step se entry par milte hain.
    Node* slow = head;
    Node* fast = head;

    while (fast != nullptr && fast->next != nullptr) {
        slow = slow->next;
        fast = fast->next->next;

        if (slow == fast) {
            slow = head;
            while (slow != fast) {
                slow = slow->next;
                fast = fast->next;
            }
            return slow;
        }
    }

    return nullptr;
}

```

Time Complexity: $O(n)$

Space Complexity: $O(1)$

15. Length of Loop in LL

Problem

Find the length of the cycle in a linked list.

Brute Approach

Store each node with the timer/index when it was first visited.

```
int lengthOfLoopBrute(Node* head) {  
    // Hint: repeated node par current timer - old timer = loop length.  
    unordered_map<Node*, int> mp;///this will store the time it visited  
    Node* temp = head;  
    int timer = 0;  
  
    while (temp != nullptr) {  
        if (mp.find(temp)) {  
            return timer - mp[temp];  
        }  
        mp[temp] = timer;  
        timer++;  
        temp = temp->next;  
    }  
  
    return 0;  
}
```

Time Complexity: $O(n)$ average

Space Complexity: $O(n)$

Better

No separate better approach needed.

Optimal Approach

Detect loop using Floyd. Once slow meets fast, keep one pointer fixed and count one full cycle.

```

int countCycleLength(Node* meetingPoint) {
    // Hint: meeting point se ek round lagao aur nodes count karo.
    int count = 1;
    Node* temp = meetingPoint->next;

    while (temp != meetingPoint) {
        count++;
        temp = temp->next;
    }

    return count;
}

int lengthOfLoopOptimal(Node* head) {
    // Hint: Floyd se meeting mile, phir cycle length count karo.
    Node* slow = head;
    Node* fast = head;

    while (fast != nullptr && fast->next != nullptr) {
        slow = slow->next;
        fast = fast->next->next;

        if (slow == fast) {
            return countCycleLength(slow);
        }
    }

    return 0;
}

```

Time Complexity: $O(n)$

Space Complexity: $O(1)$

16. Check if LL is Palindrome or Not

Problem

Check whether the linked list reads the same forward and backward.

Brute Approach

Push all values into a stack, then compare while traversing again.

```

bool isPalindromeBrute(Node* head) {
    // Hint: stack reverse order deta hai; front traversal se compare karo.
    stack<int> st;
    Node* temp = head;

    while (temp != nullptr) {
        st.push(temp->data);
        temp = temp->next;
    }

    temp = head;
    while (temp != nullptr) {
        if (temp->data != st.top()) return false;
        st.pop();
        temp = temp->next;
    }

    return true;
}

```

Time Complexity: $O(2n)$

Space Complexity: $O(n)$

Better

No separate better approach needed.

Optimal Approach

Find middle, reverse second half, compare both halves, then restore the list.

```

Node* reverseSLLRecursiveOptimal(Node* head) {
    // Hint: recursion last node ko new head bana deti hai.
    if (head == nullptr || head->next == nullptr) return head;

    Node* newHead = reverseSLLRecursiveOptimal(head->next);
    Node* front = head->next;
    front->next = head;
    head->next = nullptr;

    return newHead;
}

```

```

bool isPalindromeOptimal(Node* head) {
    // Hint: slow first half ke end par rukta hai; second half reverse karke compare.
    if (head == nullptr || head->next == nullptr) return true;

    Node* slow = head;
    Node* fast = head;

    while (fast->next != nullptr && fast->next->next != nullptr) { //first slow chah
iye hai hume
        slow = slow->next;
        fast = fast->next->next;
    }

    Node* newHead = reverseSLLIterativeOptimal(slow->next);
    Node* first = head;
    Node* second = newHead;

    while (second != nullptr) {
        if (first->data != second->data) {
            slow->next = reverseSLLIterativeOptimal(newHead); // slow->next=.....likhn
e ki jroort hi nhi h because humne link kabhi break hi nhi kiya hai; hum sirf reverse
SLLIterativeOptimal(newHead) bhi likh sakte the
            return false;
        }
        first = first->next;
        second = second->next;
    }

    slow->next = reverseSLLIterativeOptimal(newHead); // slow->next=.....likhne ki jro
rt hi nhi h because humne link kabhi break hi nhi kiya hai; hum sirf reverseSLLitera
tiveOptimal(newHead) bhi likh sakte the
    return true;
}

```

Time Complexity: $O(2n)$

Space Complexity: $O(1)$

17. Segregate Odd and Even Nodes in Linked List

Problem

Group nodes at odd positions first, followed by nodes at even positions.

Brute Approach

Store odd-position values first, then even-position values, and rewrite the list.

```
Node* oddEvenListBrute(Node* head) {
    // Hint: positions matter, values odd/even nahi.
    if (head == nullptr || head->next == nullptr) return head;

    vector<int> values;
    Node* temp = head;

    while (temp != nullptr && temp->next != nullptr) {
        values.push_back(temp->data);
        temp = temp->next->next;
    }
    if (temp != nullptr) values.push_back(temp->data);

    temp = head->next;
    while (temp != nullptr && temp->next != nullptr) {
        values.push_back(temp->data);
        temp = temp->next->next;
    }
    if (temp != nullptr) values.push_back(temp->data);

    temp = head;
    int index = 0;
    while (temp != nullptr) {
        temp->data = values[index++];
        temp = temp->next;
    }

    return head;
}
```

Time Complexity: $O(2n)$

Space Complexity: $O(n)$

Better

No separate better approach needed.

Optimal Approach

Relink odd-position nodes and even-position nodes separately.

```
Node* oddEvenListOptimal(Node* head) {
    // Hint: odd chain aur even chain alag banao, end mein odd->next = evenHead.
    if (head == nullptr || head->next == nullptr) return head;

    Node* odd = head;
    Node* even = head->next;
    Node* evenHead = head->next;

    while (even != nullptr && even->next != nullptr) { //kyoki even humesa age rahega
        to isliye
        odd->next = odd->next->next;
        even->next = even->next->next;

        odd = odd->next;
        even = even->next;
    }

    odd->next = evenHead;
    return head;
}
```

Time Complexity: $O(n)$

Space Complexity: $O(1)$

18. Remove Nth Node from the Back of the LL

Problem

Remove the nth node from the end of the linked list.

Brute Approach

Count total nodes, then delete the $(\text{length} - n + 1)$ th node from the front.

```

Node* removeNthFromEndBrute(Node* head, int n) {
    // Hint: from-end ko from-start mein convert karo using length.
    int length = 0;
    Node* temp = head;

    while (temp != nullptr) {
        length++;
        temp = temp->next;
    }

    if (length == n) {
        Node* newHead = head->next;
        delete head;
        return newHead;
    }

    int stepsToPrevious = length - n ;//jisko delte krna hai uske previous node no.
    temp = head;
    while (temp!=null) {
        stepsToPrevious --
        if (stepsToPrevious ==0){
            break;
        }

        temp = temp->next;
    }

    Node* nodeToDelete = temp->next;
    temp->next = temp->next->next;
    delete nodeToDelete;

    return head;
}

```

Time Complexity: $O(n) + O(n - k)$

Space Complexity: $O(1)$

Better

No separate better approach needed.

Optimal Approach

Move fast pointer n steps first, then move slow and fast together.

```
Node* removeNthFromEndOptimal(Node* head, int n) {  
    // Hint: fast ko n gap do; fast end par hoga to slow delete ke previous par.  
  
    Node* fast = head;  
  
    for (int i = 0; i < n; i++) {  
        fast = fast->next;  
    }  
  
    Node* slow = head;  
  
    while (fast->next != nullptr) {  
        slow = slow->next;  
        fast = fast->next;  
    }  
  
    Node* nodeToDelete = slow->next;  
    slow->next = slow->next->next;  
    delete nodeToDelete;  
  
    return head;  
}
```

Time Complexity: $O(n)$

Space Complexity: $O(1)$

19. Delete the Middle Node in LL

Problem

Delete the middle node of a linked list.

Brute Approach

Count length, then delete node at index floor of $n/2$.

```

Node* deleteMiddleBrute(Node* head) {
    // Hint: zero-based middle index n/2 hota hai.
    if (head == nullptr || head->next == nullptr) {
        delete head;
        return nullptr;
    }

    int n = 0;
    Node* temp = head;
    while (temp != nullptr) {
        n++;
        temp = temp->next;
    }

    int res = n / 2;
    temp = head;
    while (temp != nullptr) {
        res--;
        if (res == 0) {
            // i am standing before middle element;
            Node* middle = temp->next;
            temp->next = temp->next->next;
            delete middle;
            break;
        }
        temp = temp->next;
    }

    return head;
}

```

Time Complexity: $O(n + n/2)$

Space Complexity: $O(1)$

Better

No separate better approach needed.

Optimal Approach

Use slow-fast, but start fast two steps ahead so slow stops before middle.

```
Node* deleteMiddleOptimal(Node* head) {  
    // Hint: slow ko middle ke previous par rokna hai, isliye fast=head->next->next.m  
    eans aik step skip kr do  
    if (head == nullptr || head->next == nullptr) {  
        delete head;  
        return nullptr;  
    }  
  
    Node* slow = head;  
    Node* fast = head->next->next;  
  
    while (fast != nullptr && fast->next != nullptr) {  
        slow = slow->next;  
        fast = fast->next->next;  
    }  
  
    Node* middle = slow->next;  
    slow->next = slow->next->next;  
    delete middle;  
  
    return head;  
}
```

Time Complexity: $O(n/2)$

Space Complexity: $O(1)$

20. Merge two sorted linked list

function the convert array to linked list

```
Node* convertArrayToSLL(vector<int>& arr) {  
    // Hint: head fixed rahega, mover last node par move karega.  
    if (arr.empty()) return nullptr;  
  
    Node* head = new Node(arr[0]);  
    Node* mover = head;  
  
    int n = arr.size();  
    for (int i = 1; i < n; i++) {  
        Node* temp = new Node(arr[i]);  
        mover->next = temp;  
        mover = temp;  
    }  
  
    return head;  
}
```

Now store values of both th linked list in an array , sort, and the convert arrat to linked list

```

Node* mergeTwoSortedSLL(Node* head1, Node*head2) {
    // Hint: head fixed rahega, mover last node par move karega.
    vector< int>v;
    Node* temp1=head1;
    Node* temp2= head2;
    while( temp1!=nullptr){
        v.push_back(temp1->data);
        temp1=temp1->next;
    }
    while( temp2!=nullptr){
        v.push_back(temp2->data);
        temp2=temp2->next;
    }
    sort(v.begin(), v.end());

    Node* head=convertArrayToSLL(v);

    return head;
}

```

tc= $O(n_1 + n_1 + N \log N) + O(N)$; $n = n_1 + n_2$; sc= $O(N)$;

by concept of dummy Node

```
Node* mergeTwoSortedSLL(Node* head1, Node*head2) {

    Node* dummy= new Node( -1);
    Node* temp =dummy;
    Node* t1= head1;
    Node* t2= headf2;
    while(t1!=nullptr && t2!=nullptr){
        if(t1->data < t2->data){
            temp->next=t1;
            t1=t1->next;
        }
        else{
            temp->next=t2;
            t2=t2->next;
        }
        temp=temp->next;
    }
    if( t1) temp->next=t1;
    else{
        temp->next=t2;
    }
    return dummy->next;

}
```

20. Sort LL

Problem

Sort a linked list in ascending order.

Brute Approach

Store values in an array, sort, then rewrite values in linked list.

```

Node* sortLLBrute(Node* head) {
    // Hint: linked list ko array bana ke sort karna easiest brute hai.
    vector<int> values;
    Node* temp = head;

    while (temp != nullptr) {
        values.push_back(temp->data);
        temp = temp->next;
    }

    sort(values.begin(), values.end());

    temp = head;
    int index = 0;
    while (temp != nullptr) {
        temp->data = values[index++];
        temp = temp->next;
    }

    return head;
}

```

Time Complexity: $O(n \log n) + O(2n)$

Space Complexity: $O(n)$

Better

No separate better approach needed.

Optimal Approach

Use merge sort because linked list can be split and merged without random access.

```

Node* mergeTwoSortedLists(Node* first, Node* second) {
    // Hint: dummy node merge ko simple banata hai.
    Node* dummy = new Node(-1);
    Node* temp = dummy;

    while (first != nullptr && second != nullptr) {
        if (first->data < second->data) {
            temp->next = first;
            first = first->next;
        } else {
            temp->next = second;
            second = second->next;
        }
        temp = temp->next;
    }

    if (first != nullptr) temp->next = first;
    else temp->next = second;

    Node* mergedHead = dummy->next;
    delete dummy;
    return mergedHead;
}

Node* findMiddleForSort(Node* head) {
    // Hint: fast=head->next se slow first middle par rukta hai, split easy hota hai.
    Node* slow = head;
    Node* fast = head->next; // for edge case case so that slow stop at first mid

    while (fast != nullptr && fast->next != nullptr) {
        slow = slow->next;
        fast = fast->next->next;
    }

    return slow;
}

Node* sortLLOptimal(Node* head) {
    // Hint: divide list into two halves, sort both, merge.
    if (head == nullptr || head->next == nullptr) return head;

    Node* middle = findMiddleForSort(head);

```



```
Node* right = middle->next;
middle->next = nullptr;
Node* left = head;

left = sortLLOptimal(left);
right = sortLLOptimal(right);

return mergeTwoSortedLists(left, right);
}
```

Time Complexity: $O(n \log n)$

Space Complexity: $O(\log n)$, recursion stack

21. Sort a Linked List of 0s, 1s and 2s

Problem

Sort a linked list containing only 0, 1, and 2.

Brute Approach

Count 0s, 1s, and 2s; then overwrite node values.

```

Node* sort012Brute(Node* head) {
    // Hint: data replace allowed ho to counting easiest hai.
    int count0 = 0, count1 = 0, count2 = 0;
    Node* temp = head;

    while (temp != nullptr) {
        if (temp->data == 0) count0++;
        else if (temp->data == 1) count1++;
        else count2++;
        temp = temp->next;
    }

    temp = head;
    while (temp != nullptr) {
        if (count0 > 0) {
            temp->data = 0;
            count0--;
        } else if (count1 > 0) {
            temp->data = 1;
            count1--;
        } else if (count2 > 0) {
            temp->data = 2;
            count2--;
        }
        temp = temp->next;
    }

    return head;
}

```

Time Complexity: $O(2n)$

Space Complexity: $O(1)$

Better

No separate better approach needed.

Optimal Approach

Change links using three dummy lists for `0`, `1`, and `2`.

```

Node* sort012Optimal(Node* head) {
    // Hint: three chains banao: zero, one, two; end mein connect kar do.
    if (head == nullptr || head->next == nullptr) return head;

    Node* zeroHead = new Node(-1);
    Node* oneHead = new Node(-1);
    Node* twoHead = new Node(-1);

    Node* zero = zeroHead;
    Node* one = oneHead;
    Node* two = twoHead;
    Node* temp = head;

    while (temp != nullptr) {
        if (temp->data == 0) {
            zero->next = temp;
            zero = zero->next;
        } else if (temp->data == 1) {
            one->next = temp;
            one = one->next;
        } else {
            two->next = temp;
            two = two->next;
        }
        temp = temp->next;
    }

    zero->next = (oneHead->next != nullptr) ? oneHead->next : twoHead->next;
    one->next = twoHead->next;
    two->next = nullptr;

    Node* newHead = zeroHead->next;
    delete zeroHead;
    delete oneHead;
    delete twoHead;
    return newHead;
}

```

Time Complexity: $O(n)$

Space Complexity: $O(1)$

22. Find the Intersection Point of Y LL

Problem

Given two linked lists that may intersect by reference, return the intersection node.

Brute Approach

Store all nodes of first list in a hash set, then scan second list.

```
Node* intersectionBrute(Node* headA, Node* headB) {  
    // Hint: intersection value se nahi, same node address se hota hai.  
    unordered_map<Node* ,int> mp;  
  
    while (headA != nullptr) {  
        mp[headA]++;  
        headA = headA->next;  
    }  
  
    while (headB != nullptr) {  
        if (mp.count(headB)) return headB;  
        headB = headB->next;  
    }  
  
    return nullptr;  
}
```

Time Complexity: $O(n + m)$ average

Space Complexity: $O(n)$

Better Approach

Find both lengths, move longer list ahead by the length difference, then move both together.

```

int listLength(Node* head) {
    int length = 0;
    while (head != nullptr) {
        length++;
        head = head->next;
    }
    return length;
}

Node* collisionPoint(Node* smaller, Node* larger, int difference) {
    // Hint: longer list ko difference steps advance karo.
    while (difference--) {
        larger = larger->next;
    }

    while (smaller != larger) {
        smaller = smaller->next;
        larger = larger->next;
    }

    return smaller;
}

Node* intersectionBetter(Node* headA, Node* headB) {
    int n1 = listLength(headA);
    int n2 = listLength(headB);

    if (n1 < n2) return collisionPoint(headA, headB, n2 - n1);
    return collisionPoint(headB, headA, n1 - n2);
}

```

Time Complexity: $O(n + m)$

Space Complexity: $O(1)$

Optimal Approach

Use two pointers. When one reaches null, move it to the other list head.

```

Node* intersectionOptimal(Node* headA, Node* headB) {
    // Hint: switching heads makes both pointers travel equal total distance.
    if (headA == nullptr || headB == nullptr) return nullptr;

    Node* temp1 = headA;
    Node* temp2 = headB;

    while (temp1 != temp2) {
        temp1= temp1->next;
        temp2=temp2->next;
        if(temp1==temp2) return temp1;

        if( temp1==nullptr) temp1=headB;
        if(temp2==nullptr) temp2=headA;
    }

    return temp1;
}

```

Time Complexity: $O(n + m)$

Space Complexity: $O(1)$

23. Add One to a Number Represented by LL

Problem

Each node stores one digit. Add 1 to the number.

Brute Approach

Reverse the list, add carry from front, then reverse back.

```

Node* addOneBrute(Node* head) {
    // Hint: reverse karne se least significant digit front par aa jata hai.
    head = reverseSLLIterativeOptimal(head);

    Node* temp = head;
    int carry = 1;

    while (temp != nullptr) {
        int sum = temp->data + carry;
        temp->data = sum % 10;
        carry = sum / 10;

        if (carry == 0) break;

        temp = temp->next;
    }
    if (carry != 0) {
        Node* newnode = new Node(carry);
        head = reverseSLLIterativeOptimal(head);
        newnode->next = head;
        return newnode;
    }

    head = reverseSLLIterativeOptimal(head);
    return head;
}

```

Time Complexity: $O(3n)$

Space Complexity: $O(1)$

Better

No separate better approach needed.

Optimal Approach

Use recursion to reach the last digit first.

```

int addOneCarryHelper(Node* temp) {
    // Hint: carry tail se head ki direction mein return hota hai.
    if (temp == nullptr) return 1;

    int carry = addOneCarryHelper(temp->next);
    temp->data = temp->data + carry;

    if (temp->data < 10) return 0;  iske aae jana hi nhi h

    temp->data = 0;
    return 1;
}

Node* addOneOptimal(Node* head) {
    // Hint: final carry bache to new head 1 banana padega.
    int carry = addOneCarryHelper(head);

    if (carry == 1) {
        Node* newHead = new Node(1);
        newHead->next = head;
        return newHead;
    }

    return head;
}

```

Time Complexity: $O(n)$

Space Complexity: $O(n)$, recursion stack

24. Add Two Numbers in Linked List

Problem

Add two numbers represented by linked lists where digits are stored in reverse order.

Brute / Better

Converting to integer can overflow, so it is not a safe approach. Direct digit-by-digit carry is preferred.

Optimal Approach

Traverse both lists and carry just like normal addition.


```

Node* addTwoNumbersOptimal(Node* l1, Node* l2) {
    // Hint: sum = digit1 + digit2 + carry; new digit sum%10, carry sum/10.
    Node* dummy = new Node(-1);
    Node* current = dummy;
    Node* temp1= l1;
    Node* temp2= l2;
    int carry = 0;

    while (temp1 != nullptr || temp2 != nullptr ) {
        int sum = carry;

        if (temp1 ) sum += temp1->data;
        if (temp2) sum += temp2->data;
        Node* newNode = new Node(sum % 10);
        carry = sum / 10;
        current->next=newNode;
        current = current->next;
        if(temp1)temp1=temp1->next;
        if(temp2)temp2=temp2->next;
    }
    if(carry){
        Node* newNode = new Node(carry);

        current->next=newNode;
        current = current->next;
    }
    head= dummy->next;
    return head;
}

```

Time Complexity: $O(\max(n, m))$

Space Complexity: $O(\max(n, m))$, for the answer list